

VELOCITY — Project Defense Dossier

News Recommendation Engine · Java/Spring Boot + Python · Kafka · Redis · PostgreSQL · LSH

One-stop interview defense document. Structured against the interrogation taxonomy mined from the 124-document campus interview corpus (patterns A1-A6). Every answer here is grounded in the actual codebase, PROJECT_SPEC.md, and the Defensibility Tracker — nothing invented.

0. ELEVATOR PITCHES (memorize both)

10-second: “Velocity is a two-stage news recommendation engine — retrieval then ranking — built on real Microsoft MIND click data. Five microservices, Java and Python, with Kafka as the event backbone. The design goal is minimizing read latency: all heavy computation happens on the write path, so serving is just fetch-and-merge.”

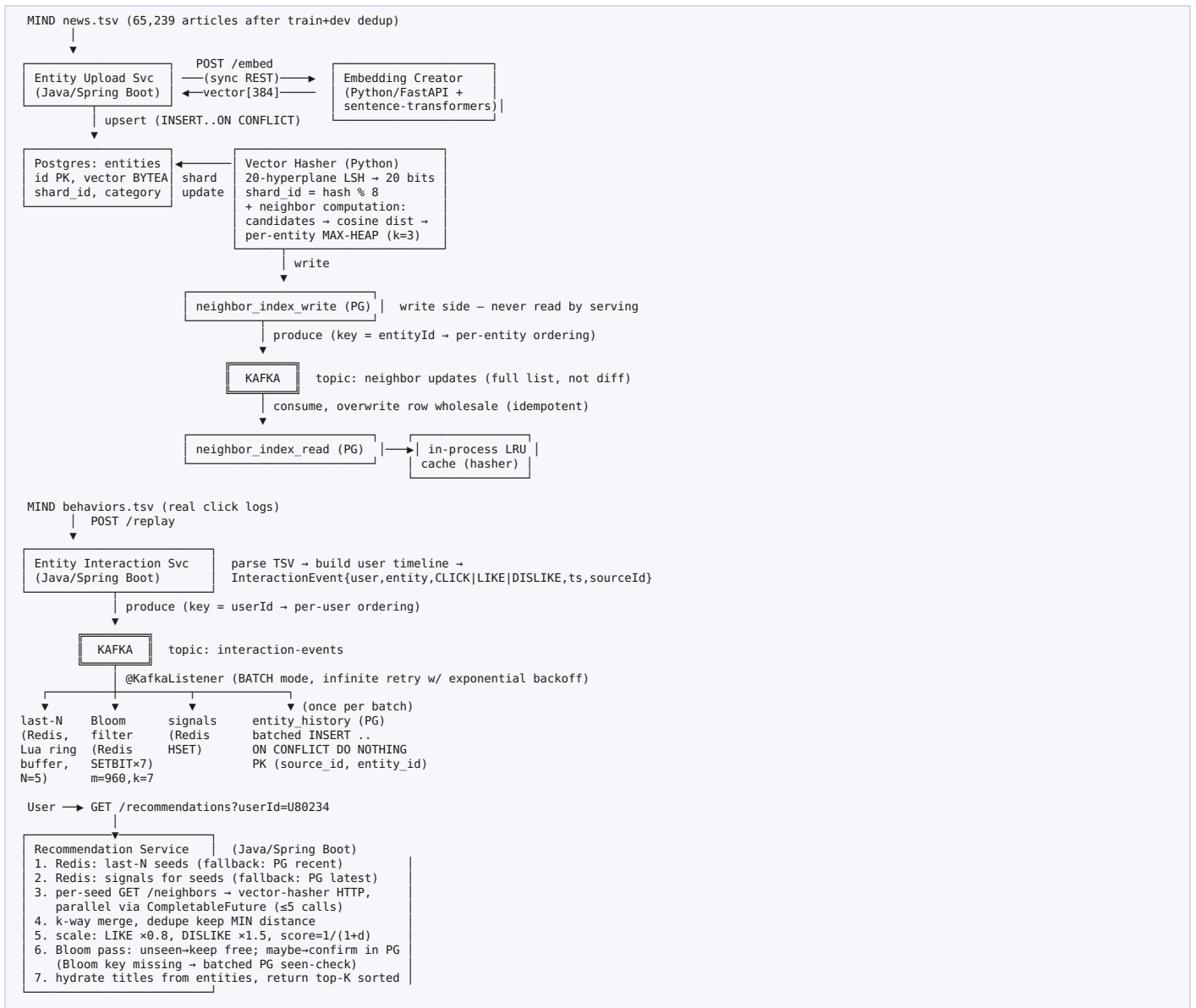
60-second: “Velocity solves the same problem YouTube/TikTok-style feeds solve, on the MIND-small dataset — 65,000 real news articles and real user click logs. Articles get embedded into 384-dim vectors, hashed with random-hyperplane LSH into shards, and each article’s nearest neighbors are precomputed into a write-side index that syncs to a read-side copy through Kafka — CQRS-style. User clicks are replayed from MIND’s real logs through Kafka into two stores: Postgres as the durable history, and Redis holding a last-N ring buffer, a Bloom filter of everything the user has seen, and like/dislike signals. The serving path takes the user’s last 5 articles, fetches their precomputed neighbor lists in parallel, k-way merges them keeping minimum distance, applies like/dislike scaling, then runs every candidate through the Bloom filter — ‘definitely unseen’ costs zero DB calls, ‘possibly seen’ gets confirmed against Postgres. Two hard guarantees: relevant articles, and never an article the user already saw. I verified it end-to-end against a real MIND user’s actual click history.”

Domain-fit framing (say this before they do): “Even though it’s a recommendation problem, Velocity is fundamentally a *backend systems* project — the ML part is a frozen pretrained model behind one FastAPI endpoint. Everything interview-relevant is systems work: Kafka event design, CQRS table separation, cache invalidation, probabilistic data structures, sharding, idempotency, and failure handling. That’s exactly the work an SDE does.”

1. SYSTEM ARCHITECTURE (be able to draw this on paper, from memory)

The corpus shows the on-paper architecture drawing is near-mandatory (EDS, UBS, Nomura, OFSS all demanded it — sometimes twice, verbally then on paper). Practice re-drawing this until it takes under 3 minutes.

1.1 Full system



1.2 The one sentence that explains the whole design

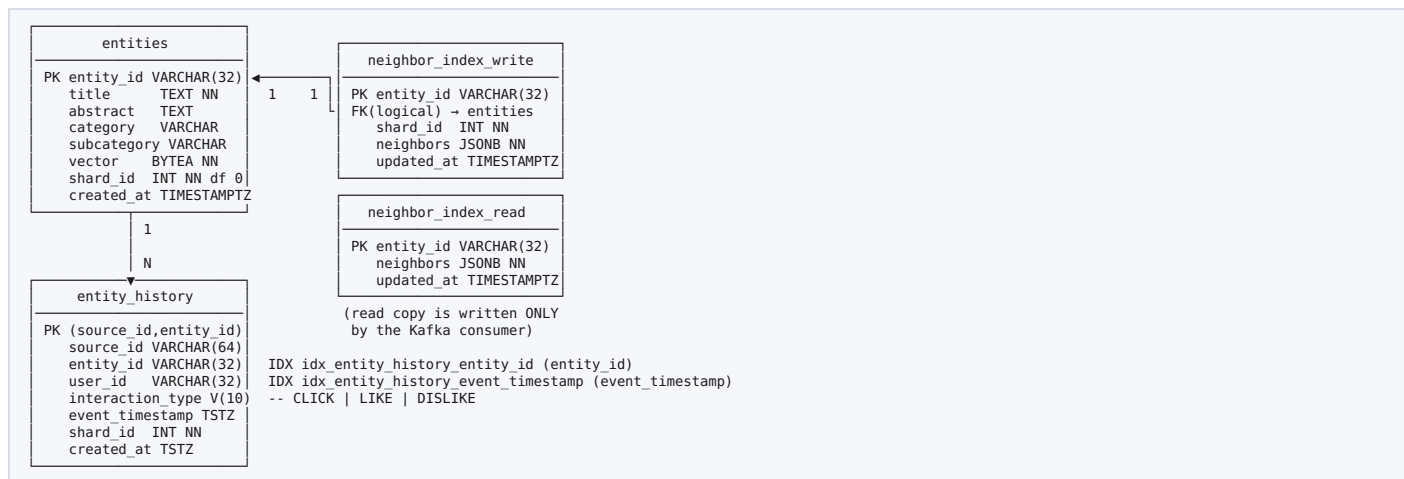
“Kafka is the only link between the produce side and the consume side — everything left of Kafka knows nothing about Postgres/Redis, everything right of Kafka knows nothing about TSVs or HTTP. That’s why the write side can fall behind or restart without losing anything: events sit on the topic until a consumer is ready.” (This exact framing is written in the repo’s REQUEST_FLOW.md — it’s your strongest “explain the architecture philosophy” line.)

1.3 North-star design principle (state it early, unprompted)

Minimize read latency by pushing computation to the write path. Neighbor lists are precomputed at write time; the serving path never touches a vector or computes a cosine distance — it only fetches precomputed lists and merges. Every architectural choice below traces back to this.

2. DATABASE & DBMS SECTION (schema, ER, SQL — corpus pattern A2 schema-design + IDFC-style escalation)

2.1 ER diagram



Important honesty note if asked “where are your FOREIGN KEYS?”: relationships are logical, not enforced by FK constraints. Defend it: (a) `entity_history` is populated by high-volume batched Kafka consumption — an FK check per row adds write-path cost against the project’s own north star; (b) the tables have different owners (upload-service owns `entities`, interaction-service owns `entity_history`, vector-hasher owns the neighbor tables) — in a microservice split these would live in separate databases where cross-DB FKs are impossible anyway; enforcing referential integrity at the application/event layer is the standard microservice pattern. **Flip condition:** in a single-owner monolith with modest write volume, I’d add the FKs.

2.2 Why each schema decision (rapid-fire ready)

Decision	Why	Rejected alternative + flip condition
<code>entity_id VARCHAR(32)</code> natural PK	MIND IDs (“N12345”) are the universal external identifier across every service and Kafka message	Surrogate <code>SERIAL</code> id — adds a join/lookup for zero benefit when the natural key is stable, short, and externally meaningful
<code>vector BYTEA</code> (1536 raw bytes = 384 × float32 LE)	No SQL-side vector math ever happens — all math is in Java/Python. <code>BYTEA</code> avoids <code>float4[]</code> ’s clunky JDBC <code>java.sql.Array</code> marshaling for a write-once/read-bulk value	<code>pgvector</code> — would undercut the entire point of the project (hand-building LSH sharding + neighbor search). Flip: if I wanted DB-side similarity search, <code>pgvector</code> immediately
<code>neighbors JSONB</code> (deliberate denormalization)	Neighbor list is always read/written wholesale as one atomic unit — the consumer overwrites the full list per Kafka message. A normalized (<code>entity_id</code> , <code>neighbor_id</code> , <code>distance</code> , <code>rank</code>) table would need delete+insert of k rows in a transaction per update, and every read is “give me the whole ordered list” anyway	Normalized child table — right answer if you needed per-neighbor queries (“who has N123 as a neighbor?”) or partial updates. This is a 1NF-violation-on-purpose talking point: know it
Composite PK (<code>source_id</code> , <code>entity_id</code>) on <code>entity_history</code>	One real MIND event = one row; replaying the same event (Kafka redelivery, manual re-run) hits ON CONFLICT DO NOTHING → idempotency at the schema level , not app level	Auto-increment PK — every replay would duplicate history and corrupt the Bloom filter’s ground truth
Two secondary indexes (<code>entity_id</code> , <code>event_timestamp</code>)	PK only covers lookups when <code>source_id</code> is known. Serving path needs “recent history by user/time” (Redis fallback) and “seen-check by entity”	No indexes — the three Redis-loss fallbacks would seq-scan
Write/read table split (CQRS)	Write side is heap-maintenance territory (constant mutation during indexing); read side is serving territory. Different sharding keys too: write shards by simhash bucket (co-locates similar entities for cheap co-update), read shards by hash(entityId) (kills hot partitions — similar-content clustering would make one shard viral)	One table — write contention and hot partitions on the serving path

2.3 “Write me a SQL query” drills (IDFC/WorkIndia pattern — they escalate on YOUR schema)

Practice these on paper against `entity_history`:

```

-- 1. Clicks per user per day (the "bookings per quarter" analog)
SELECT user_id, date_trunc('day', event_timestamp) AS day, COUNT(*) AS clicks
FROM entity_history
WHERE interaction_type = 'CLICK'
GROUP BY user_id, day
ORDER BY day;

-- 2. Escalation: % of each user's interactions that are LIKES
SELECT user_id,
       ROUND(100.0 * COUNT(*) FILTER (WHERE interaction_type = 'LIKE')
            / COUNT(*), 2) AS like_pct
FROM entity_history
GROUP BY user_id;

-- 3. Top-10 most-clicked articles with titles (JOIN)
SELECT e.entity_id, e.title, COUNT(*) AS clicks
FROM entity_history h
JOIN entities e ON e.entity_id = h.entity_id
WHERE h.interaction_type = 'CLICK'
GROUP BY e.entity_id, e.title
ORDER BY clicks DESC LIMIT 10;

-- 4. Users who clicked in ≥2 distinct categories (HAVING)
SELECT h.user_id, COUNT(DISTINCT e.category) AS cats
FROM entity_history h JOIN entities e USING (entity_id)
GROUP BY h.user_id HAVING COUNT(DISTINCT e.category) >= 2;

-- 5. Latest interaction per (user, entity) – the signals-fallback query shape
SELECT DISTINCT ON (entity_id) entity_id, interaction_type
FROM entity_history
WHERE user_id = 'U80234' AND entity_id = ANY(%)
ORDER BY entity_id, event_timestamp DESC;

-- 6. The upsert the repo actually uses (know it cold)
INSERT INTO entity_history (source_id, entity_id, user_id, interaction_type,
                          event_timestamp, shard_id)
VALUES (?, ?, ?, ?, ?, ?)
ON CONFLICT (source_id, entity_id) DO NOTHING;

-- 7. JSONB query on neighbor_index_read (shows you know JSONB isn't a blob)
SELECT entity_id, jsonb_array_length(neighbors) AS k
FROM neighbor_index_read
WHERE neighbors @> '[{"entityId": "N27132"}]';

```

2.4 Normalization talking points

- `entities` is in **3NF/BCNF**: every non-key attribute depends on the whole key (a single-column PK), no transitive dependencies. If pushed: `category`→`subcategory` is NOT a dependency here (`subcategory` belongs to the article, and a `subcategory` implies its `category` — if the interviewer catches “`subcategory` → `category`,” concede it’s a mild transitive dependency and say a `categories(subcategory PK, category)` lookup table would fix it; kept flat because the pair is display-only reference data).
- `neighbor_index` * **deliberately violates 1NF** (JSONB array = non-atomic values). Own it as a decision, per 2.2, using the “always read/written wholesale” argument. This turns a “gotcha” into your best DBMS answer.
- `entity_history` composite PK doubles as the idempotency mechanism — say “schema-level idempotency” and connect it to Kafka at-least-once delivery.

2.5 The real DB bug (tell this story — corpus rewards real bugs over clean stories)

“When I scaled ingestion from ~20 test entities to the full 65,239, vector-hasher’s shard-update query broke: it built one giant `UPDATE ... FROM (VALUES ...)` with 2 bound parameters per row — 130,478 params — and Postgres has a hard 65,535-bound-parameter limit per statement (`psycopg.OperationalError`). Fixed by chunking into batches of 10,000 assignments (20,000 params each) reusing one connection. The deeper lesson: code verified only at toy scale hides an entire class of limits — that bug lived silently in ‘completed’ Milestone 2 code until real data volume hit it.”

3. DTOs, MODELS & LAYERING (justify against the ER diagram)

The corpus shows DTO literacy gets tested directly (OFSS candidate “had never heard of DTO” — logged as a stumble).

3.1 What crosses which boundary

Type	Kind	Boundary it crosses	Backing table
InteractionEvent {userId, entityId, interactionType, timestamp, sourceId}	DTO — the one shape crossing every boundary : Kafka message value, builder output, and the row shape for Postgres	HTTP→internal, producer→Kafka→consumer, consumer→JDBC	<code>entity_history</code>
BehaviorRow	Internal model — never crosses Kafka, never leaves producer side	none (in-process only)	none (parse artifact of behaviors.tsv)
ReplayResult {usersReplayed, eventsPublished}	Response DTO — the only shape the HTTP caller ever sees	service→HTTP	none
EmbedRequest {entityId, text} / EmbedResponse {entityId, vector[384]}	Request/response DTOs — the Java↔Python API contract	upload-svc→embedding-creator (REST)	<code>entities.vector</code>
NewsArticle	Domain model of a parsed news.tsv row	parser→ingestion service	<code>entities</code>
NeighborEntry {entityId, distance} / NeighborsResponse	DTOs for vector-hasher’s GET /neighbors contract	rec-svc→vector-hasher (REST)	<code>neighbor_index_read.neighbors</code> (JSONB elements map 1:1 to NeighborEntry)

RecommendationItem {entityId, title, category, score} / RecommendationResponse	Response DTOs — the public API	rec-svc→user	joins entities + computed score
InteractionType enum (CLICK/LIKE/DISLIKE)	Shared vocabulary type, duplicated per service	—	entity_history.interaction_type

3.2 The DTO justification (one paragraph, memorize)

“DTOs exist wherever a shape crosses a process boundary — HTTP, Kafka, or another service’s API. The design rule in Velocity: internal models (BehaviorRow, NewsArticle) never leak across a boundary, and external callers never see table shapes. That’s why RecommendationItem exposes {entityId, title, category, score} — score is computed, title is hydrated from entities, and nothing about vectors, shards, or history rows leaks out. And notice InteractionType is deliberately duplicated in each service rather than shared as a library — microservices sharing a JAR for one enum couples their release cycles; duplicating three constants is the cheaper coupling.”

3.3 If asked “DTO vs Entity vs Model — difference?”

Entity = persistence-layer object mapped to a table row. DTO = transfer shape across a boundary, no behavior, shaped for the *consumer’s* needs. Model = in-process domain object with behavior. Velocity example in one line: BehaviorRow (model, has parse/classify semantics) → InteractionEvent (DTO, crosses Kafka) → entity_history row (entity).

4. A1 — TECH-STACK JUSTIFICATION BANK (“Why X over Y”)

The corpus’s single most common pattern. Every answer below is in the mandated shape: **“I used X over Y because [concrete reason tied to actual constraints], but I’d reconsider if [flip condition].”**

4.1 Kafka vs. direct synchronous calls (MasterCard asked this verbatim about Kafka)

Interaction ingestion writes to **two independent stores** (Postgres + three Redis structures). A synchronous path means the dual-write problem: if the Redis write succeeds and the Postgres write fails mid-request, the stores diverge and there’s no record to repair from. With Kafka, the event is durably accepted once; consumers apply it to each store at their own pace, and if a store is down, events wait on the topic — this was **proven live**: Redis went unreachable, the consumer retried with backoff, and when Redis returned, all 96/96 rows landed correctly with zero manual replay. Also: key=userId gives per-user ordering for free, and offset replay enabled the whole MIND-replay design. **Flip**: a single store with modest write volume and no replay need — then a transactional synchronous write is simpler and strictly better.

4.2 Why Kafka and not RabbitMQ?

Velocity needs **replay** (re-run MIND history from offset 0), **per-key ordering** (userId/entityId partitioning), and log semantics (the neighbor-index resync is CDC-style — consumers may come and go, the log stays). Kafka is a distributed log; RabbitMQ is a message broker that deletes on ack — replay and long retention aren’t its model. **Flip**: complex routing topologies or per-message priority → RabbitMQ.

4.3 KRaft vs. Zookeeper Kafka

Zookeeper mode is deprecated; KRaft is the current standard and removes an entire extra service from docker-compose. (One-liner, but knowing it signals currency.)

4.4 Redis vs. “just query Postgres” for user state

Serving-path reads (last-N, Bloom filter, signals) happen on **every** recommendation request — the north star is read latency, so these live in RAM. Redis specifically over Memcached: needed **data structures** (lists for the ring buffer, bitmaps for SETBIT, hashes for signals), **Lua scripting** (the LREM→LPUSH→LTRIM ring-buffer update runs as one atomic script — no race between two concurrent events for the same user), and built-in persistence (RDB/AOF) so a restart doesn’t wipe user state. **Flip**: if Redis were purely a look-aside cache of DB rows, Memcached would do.

4.5 Redis state loss — the fallback story (strongest reliability answer)

“I don’t *trust* Redis — all three structures have the same fallback shape: key missing → batched Postgres query → Redis self-heals on the user’s next interaction. Critically, a **missing Bloom filter key is not treated as ‘nothing seen’** — 0 bits would mean every already-seen article leaks back into recommendations. Instead it triggers a batched seen-check against entity_history for all candidates. I live-verified all three by DEleting each Redis key for a real user and confirming identical, correct results came back from Postgres.”

4.6 Bloom filter vs. a plain Redis SET of seen entityIds

A SET grows linearly with history — unbounded per-user memory. The Bloom filter is **fixed 960 bits per user forever**, O(1) membership check, and its one failure mode (false positive) is *safe by construction*: “possibly seen” just costs one confirming DB query; “definitely unseen” — the common case — costs zero DB calls. False negatives are impossible, so a seen article can never leak through. Sizing: $n=100, p=0.01 \rightarrow m = -(n \cdot \ln p) / (\ln 2)^2 \approx 960$ bits, $k = (m/n) \cdot \ln 2 \approx 7$ hashes. **Honest limit (volunteer it)**: real data showed p99 = 137 entities/user — heavy users exceed design capacity, so their false-positive rate creeps up. Safe (just more fallback DB checks), but tracked in the Defensibility Tracker; the real fix is sizing m per-user or scalable Bloom filters. **Flip**: if history were capped small (say ≤50), a plain SET is simpler and exact.

4.7 Why 2 hash calls, not 7? (Kirsch-Mitzenmacher)

The 7 bit positions are derived by double hashing: $h_i(x) = h_1(x) + i \cdot h_2(x) \bmod m$, from just two MurmurHash3 calls with different seeds. Statistically equivalent to k independent hashes for Bloom purposes, at $\sim 2/7$ ths the hashing cost. (If they push: this is a published result — Kirsch & Mitzenmacher, “Less Hashing, Same Performance.”)

4.8 Why MurmurHash3 and not MD5/SHA/hashCode()?

Non-cryptographic use case — need speed + good distribution, not preimage resistance. Java’s `String.hashCode()` has poor avalanche behavior and only 32 bits with well-known clustering; crypto hashes waste cycles. Murmur3 is the standard choice for Bloom filters/hash sharding, and implementing it identically in Java (both services) keeps bit positions consistent.

4.9 LSH vs. FAISS / pgvector / brute-force kNN

Brute force is $O(N)$ distance computations per query — 65k entities $\times$ 384 dims per insert doesn’t scale. FAISS/pgvector would *solve* the problem but *hide* it — the point of this project is demonstrating how approximate nearest neighbor works underneath: random-hyperplane LSH is the N -dimensional generalization of quadrant splitting; similar vectors agree on most hyperplane signs $\rightarrow$ similar hash $\rightarrow$ same bucket $\rightarrow$ exact cosine distance computed only within the small candidate set. **Flip:** in production I’d reach for FAISS (HNSW/IVF) immediately — better recall/latency trade-offs, battle-tested; hand-rolled LSH is pedagogical.

4.10 Why don’t you normalize the hyperplane vectors?

Deliberate: for `sign(dot(v, r))`, scaling r by any positive constant never flips the sign — normalization costs a sqrt + division per hyperplane for a mathematically identical result. Skipping it isn’t corner-cutting, it’s not doing dead work. (This exact reasoning is documented in `lsh.py`.)

4.11 Postgres vs. MongoDB (the corpus’s #1 question)

The data is relational and the access patterns are relational: joins (history $\times$ entities for titles/categories), secondary-index range scans by timestamp, `DISTINCT ON` for latest-signal, and — decisive — `INSERT ... ON CONFLICT upsert against a composite PK, which is the idempotency mechanism for at-least-once Kafka delivery`. Where the data genuinely is document-shaped (a neighbor list read/written wholesale), I used JSONB — Postgres gives document flexibility *inside* a relational engine, so I got both without operating two databases. **Flip:** genuinely schema-less entities or horizontal write scaling beyond one node’s capacity $\rightarrow$ then Mongo/Cassandra enters the conversation.

4.12 JdbcTemplate vs. Spring Data JPA

The services have a handful of queries each, and the critical one needs `native ON CONFLICT` semantics that JPA doesn’t map cleanly — `saveAll()` would be N merge-selects + inserts, not one batched idempotent statement. Raw JDBC is less machinery for well-understood statements, not more. **Flip:** many entity types with real object-graph relationships $\rightarrow$ JPA earns its complexity.

4.13 Java for services, Python for embedding + LSH — why split?

Two-sided answer (the OFSS corpus example rewarded exactly this shape): Python owns the ML ecosystem — sentence-transformers is Python, and NumPy makes hyperplane projection/cosine math trivial (`hyperplanes @ vector` is one line and vectorized in C). Java/Spring Boot owns the service layer — type safety, mature Kafka/Redis/JDBC integration, and it matches the production stack of my target roles. The split follows the workload: compute-heavy numeric code $\rightarrow$ Python/NumPy; I/O-heavy concurrent services $\rightarrow$ Java. **Flip:** if team ops burden of two runtimes outweighed it, DJL or ONNX Runtime could pull inference into the JVM.

4.14 Sync REST for /embed, but Kafka for interactions — inconsistent?

No — deliberate per-path decisions (documented in the spec): embedding happens during a **one-time batch ETL** of a static file — nothing downstream diverges if it fails; retry-and-skip is fine, decoupling buys nothing. Interactions are the **live dual-write path** where divergence is the actual risk. Async everywhere is cargo-culting; async where consistency demands it is design.

4.15 Rec-service calls vector-hasher over HTTP instead of reading neighbor_index_read directly — why the extra hop?

Single-owner rule: vector-hasher owns that table and already fronts it with an LRU cache — going direct would duplicate cache logic and create a second owner whose reads break silently if the schema evolves. `entities` and `entity_history` ARE read directly via JDBC because they’re plain shared reference/fact tables with no existing service API — same write-owner/read-elsewhere pattern `entities` already had. **Flip:** if the extra network hop showed up in latency budgets, I’d extract the neighbor-read into a shared library or move the cache to Redis.

4.16 Max-heap on the write side, flat sorted array on the read side — why different structures for the same data?

Different operations. Write side answers “is this new entity closer than my current *farthest* neighbor?” — max-heap gives $O(1)$ peek-farthest and $O(\log k)$ eviction. Read side never mutates — a flat sorted list serializes cheaply, merges cheaply, and pays zero heap-maintenance cost. Same data, two structures, each matching its access pattern — that’s the whole CQRS argument in miniature.

4.17 Why a ring buffer (last-N=5) for seeds, not full history?

Recency is the relevance signal — recommendations should follow what the user is reading *now*. Full history as seeds would (a) make request fan-out unbounded, (b) drown current interests in old ones. The Lua `LREM` $\rightarrow$ `LPUSH` $\rightarrow$ `LTRIM` keeps it $O(1)$, atomic, and deduplicated. $N=5$ caps serving fan-out at 5 parallel neighbor fetches. **Flip:** a real system would blend long-term taste vectors with recency — that’s a ranking-model concern, out of scope by design.

4.18 CompletableFuture on the common pool — no dedicated executor?

At most 5 concurrent HTTP calls per request (bounded by `LAST_N_SIZE`). A dedicated pool would be configuration for its own sake at this scale. **Flip (know it well — this is bait):** on a real service, blocking I/O on the common ForkJoinPool is dangerous — it starves parallel streams and other library users. Under real load I'd use a dedicated bounded ThreadPoolExecutor, or WebClient/virtual threads.

4.19 Infinite retry with exponential backoff instead of a dead-letter queue — isn't that wrong?

It's the deliberate opposite of the default, chosen after a real failure (full story in §7.2). A DLQ solves the **poison message** problem — a message that can *never* succeed. Every operation in this consumer is idempotent (Lua ring buffer, SETBIT, HSET, ON CONFLICT DO NOTHING), so no such message exists — only **dependencies temporarily down**, and for those the correct behavior is: wait, retry with backoff (1s initial, ×2, capped 30s), never commit offsets past unprocessed data. **Flip:** the moment a non-idempotent or validation-failing operation enters this consumer, a DLQ + ErrorHandlingDeserializer becomes necessary — and I've noted the deserializer gap as known future work.

4.20 Docker Compose, not Kubernetes?

Nine containers on one dev machine; Compose is one-command spin-up. K8s adds orchestration machinery the project doesn't exercise. The services are 12-factor (all config via env vars) so the migration path is clean. **Flip:** multi-node deployment or autoscaling needs.

5. A2 — WALKTHROUGH NARRATIVES (end-to-end, two depths each)

5.1 “Walk me through what happens when a user asks for recommendations” (THE question)

Plain version (60s): “We look up the last five articles you read. For each, we already know its most similar articles — computed ahead of time. We merge those five lists, boost things similar to what you liked, demote things similar to what you disliked, throw away everything you've already seen, and return the top ten.”

Technical version: Request hits `RecommendationService` → (1) Redis LRange on `user:{id}:lastEntities` for ≤5 seeds; if empty, fall back to `entity_history` recent query. (2) HGETALL signals for those seeds; missing key → batched latest-interaction query. (3) Five parallel `GET /neighbors/{seedId}` calls to vector-hashers via `CompletableFuture` — vector-hashers serve from their LRU cache or `neighbor_index_read`. (4) Merge all lists; a candidate appearing under multiple seeds keeps its **minimum raw distance**, and only that winning seed's signal is used for scaling (documented simplification: a closer neutral seed beats a farther LIKE seed — there's even a named test for it). (5) $\text{adjustedDistance} = \text{distance} \times \{\text{LIKE } 0.8, \text{DISLIKE } 1.5, \text{else } 1.0\}$; $\text{score} = 1/(1+\text{adjustedDistance})$; sort descending. (6) Single filter pass over the sorted list: Bloom says “definitely unseen” → keep, zero DB cost; “possibly seen” → confirm against `entity_history`; drop confirmed-seen; `limit(topK)`. (Originally sketched as split-into-two-lists-then-recombine; realized during implementation that filtering the one sorted list in place preserves score order without a second sort.) (7) Hydrate titles/categories from `entities`, return.

5.2 “Walk me through ingestion”

`news.tsv` row → `MindNewsTsvParser` → `NewsArticle` → `POST /embed {entityId, title} "+abstract"` → 384-dim vector → `EntityRepository` upsert into `entities` (batch 100, ~6 concurrent embed calls, one retry then skip-and-log per failed row). Then vector-hashers `/assign-shards: 20` hyperplane dot products → sign bits → 20-bit hash → % 8 → `shard_id` UPDATE (chunked, post-bugfix). Then `/neighbors/index-all`: per entity, fetch same-bucket candidates (widen to adjacent shards if below `min_candidates`), exact cosine distance against each, max-heap keeps top-k=3, persist to write table, produce full list to Kafka keyed by `entityId`, consumer overwrites read table row, invalidate LRU cache entry.

5.3 “Walk me through the interaction/replay path”

`POST /replay?userId=` → `ReplayService` parses both `behaviors.tsv` splits → groups rows by user → `UserTimelineBuilder` sorts chronologically, emits History events (`sourceId HIST-{userId}-{index}`) then real-click events (`sourceId = real impression ID`); LIKE/DISLIKE simulated deterministically via MurmurHash on ~10%/5% of clicks (MIND only has click/no-click — declared simulation, logged in the tracker) → producer sends each `InteractionEvent` keyed by `userId` → batch consumer: per event, Lua ring-buffer push + 7 SETBITS + HSET signal; once per batch, one JDBC batched upsert into `entity_history`.

5.4 Deliberate asymmetry to volunteer (sounds junior if they find it, senior if you present it)

Postgres keeps **every** occurrence as a distinct row (permanent record: what actually happened), while Redis last-N and the Bloom filter **collapse repeats** (they answer “what's recent” and “ever seen” — repeat counts add nothing). Not drift between stores — two different questions answered correctly. This is written in the tracker precisely so nobody “fixes” them to match.

6. A3 — SCALING / EXTENSION / HYPOTHETICAL ANSWERS (the IDFC/BlackRock/MSCI escalation pattern)

6.1 “10x/100x users tomorrow — what breaks FIRST?”

Ordered, specific answer (the corpus punishes vague “add more servers”): 1. **First real break: Redis** — single instance holding all per-user state. Fix: Redis Cluster; the design anticipated this — keys are already sharded by `hash(userId) % 8` logically, so the migration is assigning hash slots, not redesigning keys. 2. **Second: the recommendation service's synchronous fan-out** — 5 blocking HTTP calls per request on the common pool. Fix: dedicated bounded executor or WebClient/virtual threads; add a Redis-backed shared cache in front of neighbor reads (the current LRU is per-process — documented tracker entry — so horizontally scaled hasher instances wouldn't share cache state). 3. **Third: Postgres for entity_history writes** — currently batched, but single node. Fix: the `shard_id` column already encodes the partitioning scheme; physical sharding = one Postgres per `shard_id`, routing by the same `hash(userId) % N`. “The partitioning algorithm is built; distributing it across machines is ops, and that was a declared scope cut, not an oversight.” 4. **Kafka itself scales by**

partition count — topics keyed by userId/entityId already, so adding partitions + consumers in the same group is the designed path.

6.2 “100x more ARTICLES?”

Neighbor indexing cost grows with bucket density. Answers in order: raise NUM_SHARDS beyond 8 (uses more of the 20 hash bits — see the mod-8 story in §7.1); multi-probe LSH (query hash variants formed by flipping individual bits, not just ± 1 shards); at real scale, swap hand-rolled LSH for FAISS HNSW/IVF — “my architecture doesn’t change, only the candidate-generation box: everything downstream consumes {entityId, distance} lists regardless of what produced them.” That interface stability is the strongest scaling answer in the project.

6.3 “What about a viral article — hot key?”

Already designed for, twice: read-side neighbor index shards by hash(entityId) — NOT by content-similarity bucket — precisely so similar-content clustering can’t concentrate a viral topic onto one partition; and the LRU cache absorbs repeated reads of a hot entity’s list. Next steps if pushed: TTL-based cache + request coalescing (single flight) so a cache miss on a hot key doesn’t stampede the DB.

6.4 “Make it real-time — new article recommendable in seconds?”

Path already exists: ingest → embed → hash → index → Kafka → read side is event-driven end to end; the only batch-shaped part is the ETL trigger. Change: have entity-upload produce an “entity-created” event consumed by vector-hasher instead of the manual /assign-shards + /index-all calls. Cold-start caveat: a brand-new article has neighbors immediately (content-based embedding — this is exactly why content vectors beat interaction-based collaborative filtering for news, where items are born cold).

6.5 “Two clicks from the same user at the exact same moment?” (MSCI/Pulkit concurrency pattern — expect this)

Kafka key = userId → both events land on the same partition, consumed in order by one consumer thread — no cross-consumer race is possible for one user’s state. Within processing: the ring-buffer update is a single atomic Lua script (LREM→LPUSH→LTRIM can’t interleave), SETBIT is atomic per bit and idempotent, HSET last-write-wins, and the Postgres upsert is idempotent by PK. Duplicate delivery of the same event = replaying idempotent ops = no-op. “The design answer to concurrency here is: serialize per user via partitioning, make everything idempotent, and then redelivery and retries become free.”

6.6 “User clicks an article and it still shows in recommendations — what do you do?” (IDFC techno-HR pivot)

Trace the actual pipeline: click → Kafka (was it produced? check topic) → consumer lag (is the batch stuck retrying? check the WARN retry logs — I added exactly those) → Bloom filter updated? → serving read Bloom before or after? Honest framing: there IS an eventual-consistency window between click and Bloom-bit set — bounded by consumer lag, normally milliseconds. If product required read-your-own-writes, I’d have the interaction API synchronously set the Bloom bit (safe: idempotent) while Kafka still feeds the durable path — a deliberate, small dual-write for UX, with Kafka remaining the source of truth.

6.7 Business-angle questions (UBS pattern: “would a subscription model work?”)

Have one ready: “Velocity’s engine is monetization-agnostic — the ranking stage is where business logic enters. Sponsored placement = a scaling factor like the LIKE $\times 0.8$ boost, but paid; the architecture already has the exact hook (per-candidate multiplier in CandidateRanker). Subscription would tune for depth/diversity of reading; ad-funded tunes for session length. I’d A/B test via the scaling constants — they’re deliberately config-driven placeholders for exactly this reason.”

7. A6 — REAL FAILURE STORIES (what broke → wrong instinct → fix → what remains)

These are your highest-value assets. The corpus is explicit: honestly-described real bugs (MSCI/Pulkit’s concurrency gap, Baker Hughes’ OCR confession) outperform clean stories. All four below are real, logged in the Defensibility Tracker with dates.

7.1 The mod-8 LSH compression miss (observed with real data)

What: Two genuinely similar sports headlines agreed on 17 of 20 hash bits (85%) — but one of the 3 disagreeing bits was among the 3 low-order bits that decide `hash % 8`, so they landed in shards 0 and 2 — different AND non-adjacent (adjacent of 0 = {7,0,1}), so the widen-fallback couldn’t catch the pair either. Meanwhile an unrelated cooking headline coincidentally shared a shard with one of them. **Wrong instinct:** treat bucket membership as a similarity guarantee. **Understanding:** it’s an approximation by construction — mod-8 keeps only 3 bits of a 20-bit signal that itself still carried real similarity (85% agreement). That’s precisely why production ANN systems always follow bucket retrieval with **exact distance re-ranking over the candidate set** — which is exactly Milestone 3’s job in my pipeline. **At real scale:** multi-probe LSH (flip individual bits, probe those buckets) or larger NUM_SHARDS. **Why this story is powerful:** it demonstrates you understand your own algorithm’s failure mode *from data*, not from a blog post.

7.2 “Records discarded” — the Kafka data-loss incident (the crown jewel; tell it in full)

What broke: during live verification, Redis became unreachable (root cause below). Expected: consumer waits. Actual: Spring Kafka’s *default* batch error handling retried a few times, then logged `Records discarded: interaction-events-4@0..18` and **committed offsets past 19 unprocessed events** — silent data loss, not delay. The events were simply gone from processing until I manually re-replayed the user. **Wrong instinct:** assume “Kafka decouples ingestion from writes” is a property you get for free. It isn’t — the *default consumer error policy actively violates it*. **Fix:** replaced the default with `DefaultErrorHandler` over `ExponentialBackOff` (1s initial, $\times 2$, 30s cap, unlimited attempts) wired via `setCommonErrorHandler`. Deliberately declined a DLQ (no poison-message case exists — every op is idempotent) and noted `ErrorHandlingDeserializer` as separate future work (a malformed message would still crash the consumer thread — different failure mode, honestly scoped). **The bug inside the fix (tell this too):** first attempt registered a retry-logging `RetryListener` as a lambda. `RetryListener` has two `failedDelivery` overloads — the single-record one is the only abstract method (so the only one a lambda can implement), but batch listeners always invoke the *plural* `ConsumerRecords` overload, a default no-op. The lambda silently never fired.

Diagnosed via `kill -3` thread dumps proving the consumer was legitimately parked in backoff while zero log lines appeared; fixed with an anonymous class overriding the plural overload explicitly. **Verified:** twice, by `docker network disconnect/connect` to simulate the outage — batches sat retrying with WARN lines per attempt, no discards, and on reconnect the same batches flowed through with zero manual intervention (96/96 rows). One production-relevant observation from that test: the dominant inter-attempt delay was Lettuce’s ~60s default command timeout, not my 1-2s backoff, because a live connection going silent ≠ instant DNS failure. **At real scale:** keep infinite retry, add metrics/alerting on retry duration so an operator knows a dependency has been down “too long.”

7.3 The heap duplicate bug (caught by real-data verification)

`push_with_eviction` didn’t check whether an `entity_id` was already in the heap; the same entity can legitimately be pushed twice across the two `index_entity()` call paths that touch one list. Result: literal duplicate neighbor entries (confirmed in shard 7 after first bootstrap). Fix: membership check by `entity_id` before insert/evict + two regression tests in `test_heap.py`. Lesson: “my unit tests passed; *real data* found it — which is why every milestone had a real-data verify criterion, not just a green test suite.”

7.4 The cross-milestone data-availability gap

Milestones 1-3 had only ever run against ~21 entities; Milestone 4’s replay referenced thousands of MIND entity IDs with essentially zero overlap — so a real user’s seeds had no neighbor data at all. Not a Milestone 5 bug — a gap between verified-in-isolation milestones. Fix: full ingestion of train+dev news.tsv (51,282 + 42,416 → 65,239 after dedup), full shard assignment and indexing — which is what surfaced the 65,535-parameter bug (§2.5). Lesson to state: “component-level verification composes into system-level gaps; the last milestone must be an end-to-end pass at real volume.”

7.5 Environment story (use only if asked about ops/debugging)

The local Redis container had a corrupted network attachment (HostConfig claimed the network; NetworkSettings showed none — stale state from long-lived containers). Made `redis` unresolvable and masked a whole verification run. Fixed with `--force-recreate`. “Not an app bug — but it’s what triggered discovering 7.2, which very much was.”

8. A4 — OWNERSHIP DRILLDOWN PREP (per-keyword grill lists)

Corpus rule (Dolat candidate): every resume keyword has 25 follow-up variants. Minimum-competence checklist per Velocity keyword — if any line feels shaky, drill it before the interview:

Kafka: topic/partition/offset; why key ⇒ ordering (same key → same partition → one consumer); consumer groups & rebalancing; at-least-once vs at-most-once vs exactly-once (you run at-least-once + idempotent consumers — be able to say why that’s effectively exactly-once *outcome*); batch vs record listeners; offset commit timing; what KRaft replaced. **Redis:** why single-threaded is fast; Lua atomicity; SETBIT/bitmap memory model; RDB vs AOF; eviction policies; what happens on restart; TTL. **Bloom filter:** derive m and k formulas on paper; why no false negatives (bits only ever set); why deletes are impossible (counting Bloom as the fix); double hashing. **LSH:** why sign(dot) buckets similar vectors (both sides of few hyperplanes differ); why 20 hyperplanes = 20 bits; probability two similar vectors disagree on a bit ∝ angle between them; cosine vs Euclidean distance and why cosine for embeddings (direction ≈ meaning, magnitude irrelevant). **Heap:** why max-heap for k-smallest (evict the largest); O(log k) insert / O(1) peek; heapify. **Spring:** DI/IoC in one line; @Service vs @Component; how @KafkaListener containers work; where transactions would go if added. **Postgres:** how B-tree indexes serve your two secondary indexes; EXPLAIN basics; why upsert needs a unique constraint; JSONB vs JSON (binary, indexable via GIN); VACUUM in one line. **MurmurHash/hashing:** avalanche property; seed parameter; why floorMod not %.

The “manipulated input” test (BlackRock pattern — they change your inputs live): - “Embed an empty string?” → sentence-transformers still returns a 384-dim vector; it lands somewhere in space; garbage-in-garbage-neighbors — validation belongs at ingestion (skip empty title+abstract). - “A user with zero history?” → last-N empty → PG fallback also empty → no seeds → empty/limited response; honest gap: no cold-start popularity fallback — deliberate scope cut, would add “trending by category” as the empty-seed branch. - “entityId in history that’s not in entities?” → history keeps it (permanent record); serving hydration would drop it from output — LEFT-JOIN semantics question, know which yours does. - “Kafka down at replay time?” → producer send fails; replay is manually re-runnable and consumers are idempotent, so re-replay is safe by design.

9. A5 — METRICS: WHAT’S REAL (IDFC feedback: numbers must be real, not simulated)

Every number you may quote, with its measurement provenance — never claim beyond this table:

Claim	Status
65,239 distinct entities ingested, embedded, sharded, indexed	Real (train 51,282 + dev 42,416, deduped; /assign-shards confirmed for all)
384-dim sentence-transformer embeddings	Real (model property)
20-bit LSH, 8 shards, k=3 neighbors	Real config; k=3 justified: at verify-time data scale, larger k never fills candidate sets → eviction untestable; env-var, trivially raised
Bloom: 960 bits, 7 hashes, designed n=100 p=0.01; observed p99=137 entities/user	Real, including the honest over-capacity observation
96/96 events correctly recovered after simulated Redis outage	Real, verified twice
End-to-end for real user U80234: 10 correct recommendations, zero already-seen; injected click correctly excluded; identical results after each of 3 Redis key deletions; LIKE-boosted score 0.5800166... stable across fallback	Real, live-verified
LIKE ×0.8 / DISLIKE ×1.5 / 10%-5% LIKE/DISLIKE simulation	Declared placeholder/demo constants — say so unprompted
Latency numbers	You have none. Never invent one. If asked: “I didn’t benchmark latency — the north star shaped the architecture, but characterizing p99 under load is

exactly what I'd do next; the serving path is 2 Redis reads + ≤ 5 cached HTTP calls + ≤ 2 batched PG queries, so I can reason about its cost structure even without the number."

If asked "what would you measure first?": serving p50/p99, Bloom false-positive rate per user (i.e., fraction of "possibly seen" that were actually unseen), consumer lag, neighbor-cache hit rate.

10. PREDICTED CURVEBALLS WITH MODEL ANSWERS (Section-F style, Velocity-specific)

1. **"Why is this a 'recommendation engine' when there's no ML model doing the ranking?"** "Deliberate scope: the two-stage retrieval→ranking *architecture* is the industry pattern; the ranking stage's scoring function is a pluggable box. Today it's distance + signal scaling; swapping in a learned ranker changes one class (CandidateRanker) and nothing upstream. I built the system around the model slot, which is the part SDEs actually own — the model itself is a data-science deliverable."
2. **"Your Bloom filter can't delete. What if a user un-dislikes / you must forget an item (GDPR)?"** "Correct — standard Bloom filters don't support deletion; clearing bits would create false negatives for other items. Options: counting Bloom filter (increment/decrement counters instead of bits, $\sim 4\times$ memory), or periodic rebuild from entity_history — which my design gets almost free, since Postgres is the durable truth and the filter is already treated as rebuildable state (that's the fallback design). For GDPR-style deletion: delete the history rows, rebuild the filter."
3. **"Two vector-hasher instances index overlapping entities concurrently — what breaks?"** "Real gap, honestly: the in-memory heap per entity assumes one indexer. Two instances could compute divergent lists and last-write-wins on the write table. Kafka ordering by entityId protects the read side's consistency, but write-side computation isn't coordinated. Fix: partition indexing work by shard_id across instances (natural fit — shard is already the unit), or an advisory lock per entity. Single-instance was a declared scope decision."
4. **"Why full-list Kafka messages instead of diffs?"** "Idempotency and replay safety. A diff stream requires applying every message exactly once in order — miss one and you're permanently wrong. Full-state messages mean the consumer overwrites wholesale; any replay, redelivery, or restart converges to correct state. Cost: bigger messages — trivial at $k=3$, and even at $k=50$ a neighbor list is small. Classic event-carried-state-transfer vs delta trade-off."
5. **"Your LIKE/DISLIKE data is fake. Doesn't that invalidate the project?"** "MIND only has click/no-click, so I simulated 10%/5% LIKE/DISLIKE — deterministically via Murmur so replays agree — and logged it as a declared extension in the Defensibility Tracker. It demonstrates the *ranking pipeline mechanics* (signal storage, retrieval, scaling, fallback), not a claim about organic preference data. The clicks themselves — the data that matters — are 100% real."
6. **"Explain exactly what happens byte-level when you store a vector."** "384 float32s packed little-endian, no header — 1,536 bytes into BYTEA. NumPy writes it (`astype(np.float32).tobytes()` semantics), Java reads it with ByteBuffer in little-endian order. The endianness contract is documented in the schema file because it's exactly the kind of silent cross-language corruption bug that's brutal to debug."
7. **"Which single component would you rewrite first, and why?"** Pick one and mean it: "The vector-hasher's consumer thread — fire-and-forget daemon with no graceful shutdown or offset monitoring, fine for a demo, first thing I'd harden. Second: replace hand-rolled LSH with FAISS behind the same /neighbors contract." (Having a ready answer here reads as ownership; hesitation reads as unfamiliarity.)
8. **"What did Claude Code / AI write vs. you?"** (increasingly asked; IDFC corpus explicitly warns about AI-built CRUD) "I used Claude Code as an implementation pair with a strict discipline: spec first (PROJECT_SPEC.md is mine), plan-mode approval per milestone, real-verification demanded per change, and every trade-off logged by hand in a Defensibility Tracker. The architecture, the milestone decomposition, every 'why X over Y' in this conversation, and the debugging of the real failures — the Records-discarded incident, the heap duplicate, the parameter-limit bug — are things I can defend to arbitrary depth precisely because I made those calls. Ask me anything in the codebase." — then let them.
9. **"Why 8 shards? Why 20 hyperplanes? Why N=5?"** All config-driven, none hardcoded (spec rule). 20 bits: enough resolution that similar vectors share long prefixes without buckets going empty at 65k entities. 8 shards: sized to the dataset so buckets stay populated (7 of 8 populated with real data). N=5: caps serving fan-out. "The honest answer is these are reasoned starting points, not tuned optima — tuning them against recall/latency measurements is listed future work."
10. **"Is your system eventually consistent? Where exactly?"** Name every window precisely — this question separates bands: (a) click → Redis/PG visible: consumer lag. (b) neighbor write-side → read-side: Kafka propagation. (c) LRU cache → invalidated on consume, but only in the one process. All bounded, all converge, and the serving path degrades safely in each window (worst case: a just-seen article appears once more, or a slightly stale neighbor list serves). "Strong consistency exists exactly where it must: entity_history's idempotent upsert."

11. DOMAIN-FIT & ROLE-TAILORING LINES (per company type)

- **Bank/NBFC (IDFC, InCred, Barclays, UBS):** lead with entity_history — "durable, idempotent, auditable event record with exactly-once *outcomes* over at-least-once delivery — the same correctness problem as transaction ledgers." The Records-discarded story is your showpiece: silent data loss found and fixed.
- **Data/analytics-leaning (MSCI, Nomura CDO):** lead with the pipeline: embeddings → LSH → ANN retrieval → ranking; the mod-8 story shows data-driven algorithm understanding.
- **Product/backend (OFSS, MasterCard, ISS):** lead with CQRS, caching layers, Kafka event design, the serving path's latency budget.

- **If told “we don’t use AI/ML here” (ISS pattern):** “Perfect — neither does 90% of Velocity. Strip the one embedding endpoint and everything I’d talk about is Kafka, Postgres, Redis, and Java service design.”

12. “I DON’T KNOW” SCRIPTS (pre-decided, per corpus advice — practice the exact sentences)

- Full unknown: “I haven’t worked with that — my closest adjacent experience is ___, and here’s how I’d reason from there...”
- Half-known: “I know the concept but haven’t used it hands-on; in Velocity the analogous thing I *did* use is ___.”
- Caught gap in own project: “That’s a real gap — here’s what I’d do about it: ___” (never defend a hole; convert it to a fix, like the tracker does).
- Sensing you’re wrong mid-answer: stop, say “actually, let me correct myself” — never argue the tide (Quantiphi corpus warning).

13. PRE-FLIGHT CHECKLIST (Velocity-specific, night before)

1. Re-draw §1.1 and the ER diagram on paper from memory, once, timed.
2. Rehearse the three walkthroughs (§5.1–5.3) out loud — plain version first, technical on request.
3. Re-read §7.2 (Records discarded) until you can tell it in 90 seconds including the RetryListener sub-bug.
4. Re-derive Bloom sizing math (m, k formulas) and the LSH sign-bit argument on paper.
5. Write fresh (don’t reuse): one tech choice you’d defend, one bug you’re proud of, one honest weakness — pick per company from §11.
6. Verify `docker-compose up + GET /recommendations?userId=U80234` still returns results, and the GitHub repo renders, right before leaving.
7. Prepare the closing question tied to *their* stack: e.g., “You mentioned you run Kafka — do you run per-key ordering guarantees like I did, or partition differently?” (corpus: specific closing questions beat generic ones).
8. Decide your bait: introduction should plant “event-driven pipeline on real click data with some interesting failure stories” — the corpus shows interviewers take the bait.

14. STAR STORIES (pre-packaged, 90 seconds each)

S1 — Silent data loss (reliability): Situation: verifying replay, Redis unreachable. Task: guarantee no event loss. Action: discovered Spring Kafka’s default discards batches after limited retries and commits offsets past them; replaced with infinite exponential backoff; hit and diagnosed a silent RetryListener overload bug via thread dumps. Result: verified twice — outage → retry → auto-recovery, 96/96 rows, zero manual replay.

S2 — Toy scale to real scale (rigor): Situation: milestones verified at ~20 entities; full MIND replay needed 65k. Task: close the gap. Action: full dual-split ingestion; hit Postgres’s 65,535-parameter limit; chunked batching fix; re-indexed everything. Result: real user’s end-to-end recommendations, all fallbacks live-verified.

S3 — Algorithm honesty (depth): Situation: two similar headlines landed in non-adjacent shards. Task: understand, not paper over. Action: traced it to mod-8 keeping 3 of 20 bits, one unlucky bit; documented; connected to why ANN systems re-rank with exact distances. Result: a defensible, data-grounded account of the algorithm’s approximation boundary and its production fixes (multi-probe LSH).

Corpus cross-reference: §4↔A1, §5↔A2, §6↔A3, §8↔A4, §9↔A5, §7↔A6, §11↔domain-fit defusal, §12↔“handling unknowns” DOs, §13↔pre-flight checklist. The Defensibility Tracker in PROJECT_SPEC.md §10 is the primary source for §7 and §9 — keep updating it.